ShopperHub

Software Requirements Document

Version: 1.0 | Date: May 2026 | Status: Draft

Project: Full-Stack E-Commerce Platform

Repository: perfactprachi-beep/ShopperHub

CONFIDENTIAL

Table of Contents

1. Project Overview

1.1 Project Name

ShopperHub (internally: ShoppersStop Clone)

1.2 Purpose

ShopperHub is a full-stack, multi-category fashion and lifestyle e-commerce platform enabling customers to browse products, manage carts, place orders via online payment or cash-on-delivery, track deliveries, and redeem loyalty points. Administrators manage inventory, orders, banners, coupons, CMS pages, and store operations through a dedicated admin panel.

1.3 Tech Stack

Layer	Technology
Frontend	React 18, Vite 5, Tailwind CSS, Zustand, React Router v6
Backend	Node.js (ESM), Express 5
Database	PostgreSQL (via `pg` pool)
Payments	Razorpay
SMS	Fast2SMS (India), Twilio (international)
Auth	JWT (access + refresh tokens), bcrypt
File Storage	Local disk via Multer + Sharp (image processing)
PWA	Vite PWA Plugin + Workbox
Security	Helmet, CORS, express-rate-limit

1.4 Architecture Overview

Layered monolith with clear separation: React SPA → Vite proxy → Express REST API → PostgreSQL. Frontend state managed via Zustand stores. Backend follows Routes → Controllers → Services/Queries pattern.

2. Functional Requirements

Authentication

ID	Description	Input	Output	Business Logic
FR-001	User Registration	full_name, email, phone, password	JWT tokens + user	bcrypt hash; issue access + refresh tokens; httpOnly cookie
FR-002	User Login	email, password	JWT tokens + user	Verify hash; issue tokens; rate-limited 10 req/min
FR-003	Token Refresh	Refresh token (cookie)	New access token	Verify refresh token; issue new access token
FR-004	User Logout	Auth header	Success	Invalidate session
FR-005	OTP Verification	phone, OTP	Verified status	Fast2SMS (India) or Twilio (international)
FR-006	Admin Login	email, password	JWT with role=admin	Validates role='admin'

Product Catalogue

ID	Description	Input	Output	Business Logic
FR-007	List Products	category, brand, gender, price, discount, sort, page	Paginated list	Recursive CTE for subcategory inclusion
FR-008	Product Detail	slug	Product + images + variants + attributes	Parallel queries for related data
FR-009	Search Products	query string	Ranked list	PostgreSQL full-text search + ILIKE fallback
FR-010	Filter Products	price range, discount, brand, category, gender	Filtered list	Dynamic parameterized WHERE clause
FR-011	Sort Products	sort param	Sorted list	price_asc, price_desc, newest, discount, random
FR-012	Product Variants	product_id	sizes, colors, stock	Each variant has independent stock

Cart

ID	Description	Input	Output	Business Logic
FR-013	Add to Cart	productId, variantId, quantity	Updated cart	Upsert — increment if exists; validate stock
FR-014	View Cart	userId	Items with line totals	Join with product/variant/image data

FR-015	Update Quantity	cartItemId, quantity	Updated item	Validate stock before update
FR-016	Remove Item	cartItemId	Success	Hard delete from cart_items
FR-017	Clear Cart	userId	Success	Auto-triggered post-order fulfilment
FR-018	Guest Cart Merge	guestItems array	Merged cart	On login, merge local storage cart into DB

Checkout & Orders

ID	Description	Input	Output	Business Logic
FR-022	Create Razorpay Order	addressId, couponCode, usePoints, deliveryType	Razorpay orderId + amount	Batch stock check; coupon validation; points cap 20%
FR-023	Verify Payment	orderId, paymentId, signature	orderId	HMAC-SHA256 verification; atomic DB transaction
FR-024	Create COD Order	addressId, couponCode, usePoints, deliveryType	orderId	Skips Razorpay; payment_status='pending'
FR-025	Shipping Calculation	deliveryType, subtotal	Shipping amount	Standard: free ≥ ₹999 else ₹99; Express: ₹199; Pickup: ₹0
FR-026	Order Fulfilment	order data + items	Confirmed order	BEGIN → insert order → insert items → lock+deduct stock → award points → mark coupon → clear cart → COMMIT
FR-027	Cancel Order	orderId	Cancelled order	Only pending/confirmed; restocks variants; deducts earned points
FR-028	Order History	userId, page, limit	Paginated orders	Aggregated JSON items per order
FR-029	Order Detail	orderId	Order + items + address	Validates ownership
FR-030	Order Tracking	orderId	Status + expected delivery	Returns delivery_type, status, expected_by

Loyalty Points, Coupons & Addresses

ID	Description	Business Logic
FR-031	Earn Points	1 point per ₹100 spent; awarded in fulfilment transaction
FR-032	Redeem Points	Max 20% of post-coupon subtotal; 1 point = ₹1
FR-033	Points Balance	Returned in user profile
FR-034	Apply Coupon	Validates: active, not expired, min_order met, per-user + global usage limits
FR-036	Add Address	Max per user enforced; ownership validated
FR-038	Delete Address	Soft delete (is_deleted flag) — never hard-deleted

Delivery

ID	Description	Business Logic
FR-040	Express Delivery Check	Pincode lookup in express_delivery_pincodes table
FR-041	Store Pickup	Active stores with address + timing
FR-042	Store Pickup PIN	4-digit PIN generated at order creation

Admin Panel

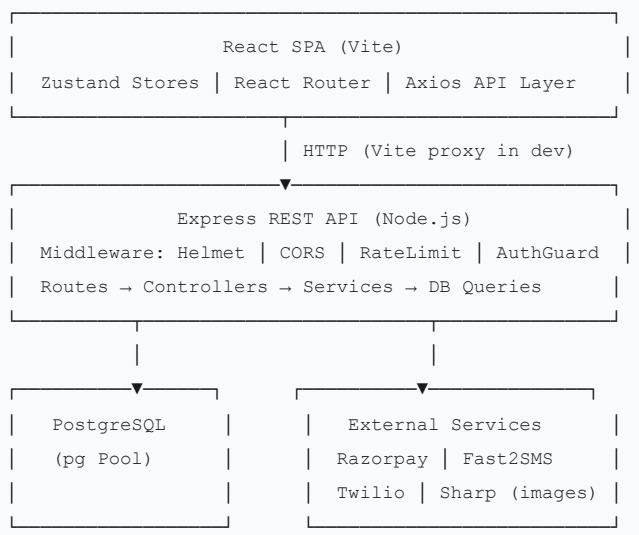
ID	Description	Business Logic
FR-049	Dashboard Stats	Parallel DB queries; 30-day sales trend
FR-050	Product CRUD	Create/update/soft-delete; image upload via Multer + Sharp
FR-051	Variant Management	Per-product; independent stock per variant
FR-052	Category CRUD	Hierarchical (parent/child)
FR-053–055	Brand / Banner / Coupon CRUD	Full CRUD with validation
FR-057	Order Management	Status transitions trigger in-app notifications
FR-058	User Management	Cannot modify/delete admin accounts
FR-059	Inventory Dashboard	Warehouse-aware; low stock alerts
FR-061	CMS Pages	Admin-managed content at /pages/:slug
FR-062–063	Store & Pincode Management	CRUD for pickup stores and express pincodes

3. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Security	All passwords hashed with bcrypt (min 10 rounds)
NFR-002	Security	JWT access tokens short-lived; refresh tokens in httpOnly cookies
NFR-003	Security	Helmet CSP: only Razorpay, Google Fonts, and self allowed
NFR-004	Security	Auth endpoints rate-limited to 10 requests/minute per IP
NFR-005	Security	Global rate limit: 1000 requests per 15 minutes per IP
NFR-006	Security	Payment signature verified server-side via HMAC-SHA256
NFR-007	Security	Admin routes protected by `authGuard` + `adminGuard` (role check)
NFR-008	Performance	All API responses gzip-compressed via compression middleware
NFR-009	Performance	DB connection pooling via pg.Pool — no per-request connections
NFR-010	Performance	Stock pre-check uses single batch ANY($1) query — no N+1
NFR-011	Performance	Product images processed/resized via Sharp before storage
NFR-012	Reliability	DB startup retries up to 5 times with exponential backoff
NFR-013	Reliability	Order fulfilment wrapped in DB transaction — atomic, rollback on failure
NFR-014	Reliability	unhandledRejection + uncaughtException handlers prevent silent crashes
NFR-015	Scalability	Stateless JWT auth — horizontally scalable (no server-side sessions)
NFR-016	Scalability	Config fully externalised via .env — 12-factor compliant
NFR-017	Scalability	Health (/api/health) and readiness (/api/ready) endpoints for orchestration
NFR-018	Maintainability	Layered architecture: routes → controllers → services → queries
NFR-019	Maintainability	ESM modules throughout — no CommonJS mixing
NFR-020	Availability	PWA with Workbox offline caching for static assets and Google Fonts
NFR-021	Testing	No automated tests present — critical gap before production

4. System Architecture

4.1 Component Diagram



4.2 Checkout Data Flow

```
User selects address/delivery
      → POST /api/payments/create-order
      → Batch stock check (ANY query)
      → Coupon validate
      → Points calc
      → Razorpay order created
      → Frontend opens Razorpay modal
      → Payment success callback
      → POST /api/payments/verify
      → HMAC-SHA256 verify
      → fulfillOrder (DB transaction: BEGIN…COMMIT)
      → SMS confirmation (fire-and-forget)
      → Response with orderId
```

4.3 External Integrations

Service	Purpose	Config Keys
Razorpay	Online payment processing	RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET
Fast2SMS	SMS for Indian numbers (+91)	FAST2SMS_API_KEY
Twilio	SMS for international numbers	TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE
Google Fonts	Typography (CSP allowlisted)	CDN, no key required

5. Data Requirements

5.1 Core Entities

Entity	Key Fields	Relationships
`users`	id, full_name, email, phone, password_hash, role, first_citizen_points, is_active	has many orders, addresses, cart_items, wishlists, reviews
`products`	id, title, slug, brand_id, category_id, base_price, discount_pct, gender, stock, status, is_deal, is_returnable	belongs to brand, category; has many variants, images, attributes
`product_variants`	id, product_id, size, color, sku, stock, extra_price	belongs to product; referenced in cart, order_items, inventory
`categories`	id, name, slug, parent_id	self-referential (parent/child); recursive CTE for querying
`orders`	id, user_id, address_id, coupon_id, subtotal, discount, shipping, total, points_earned, payment_method, payment_status, status, delivery_type, delivery_method, store_id, pickup_pin, expected_by	belongs to user, address; has many order_items
`order_items`	id, order_id, product_id, variant_id, quantity, unit_price	belongs to order, product, variant
`addresses`	id, user_id, label, line1, city, state, pincode, is_deleted	belongs to user; soft-deletable
`coupons`	id, code, discount_type, discount_value, min_order, max_uses, used_count, per_user_limit, expires_at, is_active	referenced in orders
`inventory`	id, variant_id, warehouse_id, stock_quantity	mirrors product_variants stock
`inventory_logs`	id, inventory_id, action_type, quantity, notes, created_at	audit trail for stock changes
`stores`	id, name, city, address, pincode, timing, is_active	pickup locations
`notifications`	id, user_id, message, read_at, created_at	belongs to user
`pages`	id, title, slug, content, is_active	CMS pages

5.2 Key Constraints

Field	Constraint
products.slug	UNIQUE
users.email	UNIQUE
coupons.code	UNIQUE
orders.status	ENUM: pending, confirmed, shipped, delivered, cancelled
orders.payment_status	ENUM: pending, paid, failed
orders.delivery_type	ENUM: standard, express, store_pickup

product_variants.stock	Non-negative (GREATEST constraint on deduction)
users.first_citizen_points	Non-negative (GREATEST constraint)
addresses.is_deleted	Soft delete — never hard-deleted

6. User Roles & Permissions

6.1 Roles

Role	Description
Guest	Unauthenticated visitor
Customer	Registered, authenticated user
Admin	Platform administrator (role = 'admin' in DB)

6.2 Access Matrix

Feature	Guest	Customer	Admin
Browse products/categories	✅ Yes	✅ Yes	✅ Yes
Search products	✅ Yes	✅ Yes	✅ Yes
Add to cart	Local only	✅ DB	❌ No
Add to wishlist	❌ No	✅ Yes	❌ No
Checkout / Place order	❌ No	✅ Yes	❌ No
View order history	❌ No	Own only	All orders
Cancel order	❌ No	Own (pending/confirmed)	✅ Yes
Submit review	❌ No	Purchased only	❌ No
Admin dashboard	❌ No	❌ No	✅ Yes
Product / Variant CRUD	❌ No	❌ No	✅ Yes
Order status update	❌ No	❌ No	✅ Yes
User management	❌ No	❌ No	✅ Yes
Inventory management	❌ No	❌ No	✅ Yes
CMS page management	❌ No	❌ No	✅ Yes
Payment method config	❌ No	❌ No	✅ Yes

7. API / Interface Requirements

7.1 Base URL & Response Envelope

Base URL (dev): `http://localhost:5000/api`

All responses: `{ success: boolean, data?: any, message?: string }`

7.2 Auth Endpoints

Method	Endpoint	Auth	Description
POST	/auth/register	None	Register user
POST	/auth/login	None	Login
POST	/auth/refresh	Cookie	Refresh access token
POST	/auth/logout	Bearer	Logout
POST	/auth/send-otp	None	Send OTP via SMS
POST	/auth/verify-otp	None	Verify OTP

7.3 Product Endpoints

Method	Endpoint	Auth	Description
GET	/products	None	List with filters/sort/pagination
GET	/products/:slug	None	Product detail
GET	/products/search?q=	None	Full-text search

7.4 Cart Endpoints

Method	Endpoint	Auth	Description
GET	/cart	Bearer	View cart
POST	/cart	Bearer	Add item
PUT	/cart/:id	Bearer	Update quantity
DELETE	/cart/:id	Bearer	Remove item
POST	/cart/merge	Bearer	Merge guest cart on login

7.5 Payment Endpoints

Method	Endpoint	Auth	Description
GET	/payments/methods	None	List active payment methods
POST	/payments/create-order	Bearer	Create Razorpay order

POST	/payments/create-cod-order	Bearer	Place COD order
POST	/payments/verify	Bearer	Verify payment + fulfil order
GET	/payments/orders/:id	Bearer	Fetch order by ID

7.6 Admin Endpoints (Bearer + admin role required)

Method	Endpoint	Description
GET	/admin/dashboard/stats	Dashboard metrics
GET/POST/PUT/DELETE	/admin/products	Product CRUD
GET/POST/PUT/DELETE	/admin/categories	Category CRUD
GET/POST/PUT/DELETE	/admin/brands	Brand CRUD
GET/POST/PUT/DELETE	/admin/banners	Banner CRUD
GET/POST/PUT/DELETE	/admin/coupons	Coupon CRUD
GET/POST/PUT/DELETE	/admin/offers	Offer CRUD
GET	/admin/orders	List all orders
PUT	/admin/orders/:id/status	Update order status
GET/PATCH/DELETE	/admin/users	User management
GET/POST/PUT/DELETE	/admin/payment-methods	Payment config
GET/POST/PATCH/DELETE	/admin/delivery/stores	Store management
GET/POST/PATCH/DELETE	/admin/delivery/pincodes	Pincode config
GET/POST/PUT/DELETE	/admin/pages	CMS pages

7.7 Standard HTTP Status Codes

Code	Meaning
200	Success
201	Created
400	Bad request / validation failure
401	Unauthenticated
403	Forbidden (wrong role)
404	Resource not found
409	Conflict (out of stock, duplicate)
503	Service unavailable (DB down)

8. Assumptions & Constraints

8.1 Assumptions

ID	Assumption
A1	Currency is Indian Rupees (INR) exclusively; Razorpay amounts are in paise (×100)
A2	Phone numbers with +91 prefix use Fast2SMS; all others use Twilio
A3	One user can have multiple saved addresses; all are soft-deleted, never hard-deleted
A4	A product review requires a delivered order containing that product
A5	Loyalty points have no expiry — permanent until redeemed or order cancelled
A6	Express delivery availability is pincode-gated via database config
A7	Store pickup generates a 4-digit PIN at order creation
A8	Image uploads are stored on local disk (/uploads) — no cloud storage configured

8.2 Constraints

ID	Constraint
C1	No automated test suite — all testing is currently manual
C2	No API versioning (/api/v1/) — breaking changes affect all clients immediately
C3	Local file storage does not scale horizontally — S3/CDN migration needed for multi-server deployment
C4	No refresh token revocation store — logout does not truly invalidate tokens until expiry
C5	Migration scripts are numbered manually with no version tracking tool
C6	No structured logging — all output via console.log/error; no log aggregation support
C7	Razorpay is the only online payment gateway; no PayPal, UPI QR, or wallet support

9. Glossary

Term	Definition
First Citizen Points	Loyalty reward points earned at 1 point per ₹100 spent; redeemable at checkout (1 point = ₹1, max 20% of order)
Variant	A product SKU with specific size/color combination and its own stock count
Fulfilment	The atomic DB transaction that creates an order, deducts stock, awards points, marks coupon used, and clears cart
Razorpay Order	A pre-payment object created on Razorpay's servers representing the intended transaction amount
HMAC Verification	Server-side payment signature check using SHA-256 to confirm Razorpay payment authenticity
Store Pickup	Delivery option where customer collects from a physical store using a PIN
Express Delivery	Pincode-gated premium delivery at ₹199 flat; availability stored in express_delivery_pincodes table
Soft Delete	Marking a record as inactive (is_deleted=true) instead of removing it from the database
authGuard	Express middleware that validates JWT Bearer token on protected routes
adminGuard	Express middleware that enforces role='admin' on administrative routes
CMS Page	Admin-managed static content page rendered at /pages/:slug on the frontend
Inventory Log	Audit trail entry recording stock changes (order deduction, return restock, manual adjustment)
lineTotal	Cart item total = unitPrice × quantity (computed in DB query)
paise	Indian sub-unit of currency (1 INR = 100 paise); Razorpay requires amounts in paise
Pickup PIN	4-digit random code generated at store-pickup order creation